TIH

Tablero de Información de Hidrocarburos

Documento de traspaso

Todo lo necesario para recibir y mantener el proyecto

Proyecto	TIH — Tablero de Información de Hidrocarburos
En producción	`http://172.16.83.21:8091`
Repositorio	`172.16.196.12/mvazquez/tih.git`
Entrega	Mariana Vázquez (`mvazquez`)
Fecha	Septiembre de 2026

Este documento no contiene contraseñas ni llaves. La sección 3 indica qué credenciales existen, dónde viven y por qué canal solicitarlas.

Contenido

1. Qué es TIH 3

2. Cosas que estaban locales 4

3. Accesos y credenciales 6

4. Arquitectura 7

5. Arrancar en local 8

6. Base de datos 9

7. Pendientes abiertos 10

8. Despliegue 11

9. Mapa del código 13

10. Guía de la documentación existente 14

11. Primeros pasos para quien recibe 15

1. Qué es TIH

Tablero de Información de Hidrocarburos — sitio web de la Secretaría de Energía para consultar estadísticas de hidrocarburos en México: producción, reservas, contratos y asignaciones, actividad de perforación, licitaciones, inversiones y precios de referencia.

Es la modernización del SIH anterior. Consume una base de datos SQL Server que

otro equipo alimenta; TIH casi no escribe en ella (ver §6).

En producción	http://172.16.83.21:8091
Repositorio	`https://172.16.196.12/mvazquez/tih.git` (GitLab interno)
Rama de trabajo	`master` — se commitea y empuja directo, no se usan ramas ni PRs

2. Cosas que estaban locales

Cuatro piezas del proyecto vivían en la computadora o en la cuenta personal de quien lo desarrolló. Conviene reubicarlas, pero **no todas son urgentes**:

#	Qué era	Si no se atiende
2.1	El servicio corre con `User=mvazquez`	Nada de inmediato, pero queda una trampa — ver abajo
2.2	El puente a PostgreSQL, en su PC	**Nada.** El sitio no lo usa — ver abajo
2.3	El repositorio bajo su namespace	**Se pierde el código y su historia**
2.4	Su llave SSH en el servidor	Riesgo de acceso, no de servicio

Lo único con urgencia real es **2.3**. El resto conviene ordenarlo, pero nada se cae el día que la cuenta corporativa se desactive.

2.1 El backend de producción corre con una cuenta personal

En Linux, un servicio se ejecuta "como" algún usuario del sistema. El de TIH está configurado con `User=mvazquez`, y además toda la carpeta `/opt/tih` le pertenece a esa cuenta.

```
systemctl cat tih_backend
```

> **No se rompe cuando desactiven su cuenta corporativa.** `mvazquez` en este servidor es una cuenta **local** (está en `/etc/passwd`, UID 1011) y la máquina **no está integrada al dominio** (`passwd: files systemd`, sin sssd ni winbind). Desactivar el usuario de Windows no la toca: el servicio sigue corriendo igual.

La trampa aparece después: si algún día el administrador del servidor hace limpieza de cuentas de personal que ya no está y borra `mvazquez` **del servidor**, entonces sí el servicio deja de arrancar —`User=` apuntaría a alguien que no existe— y `/opt/tih` queda huérfano, propiedad de un UID sin dueño. Nadie lo relacionaría con TIH en ese momento.

Aparte de eso, es mala práctica: producción corriendo bajo la cuenta personal de alguien que ya no trabaja ahí, y a la que quien reciba quizá no tenga acceso.

Qué hacer (sin prisa, pero antes de que alguien toque las cuentas del servidor): crear un usuario de servicio dedicado, p. ej. `tih`, cambiar `User=` en `/etc/systemd/system/tih_backend.service`, hacer `chown -R tih:tih /opt/tih` y recargar:

```
sudo systemctl daemon-reload && sudo systemctl restart tih_backend
```

2.2 El puente a PostgreSQL corría en su computadora

> **Que se detenga no afecta al sitio en producción.** El backend productivo lee **directo de SQL Server** `172.16.200.99\UMBRACODB`, no de PostgreSQL. Si el puente deja de correr, lo único que pasa es que la base de PostgreSQL —que hoy solo sirve para *probar* la migración— se queda con datos viejos. Nadie en el sitio lo nota, porque nadie la consulta.

Hay una tarea programada de Windows, registrada en la máquina de Mariana, que cada día sincroniza SQL Server -> PostgreSQL:

- Nombre: `TIH_Puente_SQLServer_a_Postgres`

- Ejecuta: `postgres_migration/run_bridge.py`
- Última corrida verificada: 30/sep/2026 10:14, resultado `0` (éxito)

Qué hacer: nada con prisa. Solo tiene sentido reactivarlo cuando se retome la migración (§7.1), y en ese momento lo correcto es montarlo en un servidor, no en una computadora personal. Para registrarlo en otra máquina mientras tanto: `postgres_migration/setup_bridge_task.ps1`.

Si más adelante se decide **cortar a PostgreSQL de verdad**, ahí sí el puente pasa a ser crítico. Hoy no lo es.

2.3 El repositorio está bajo su namespace

`https://172.16.196.12/mvazquez/tih.git` — si se borra la cuenta de GitLab, se va el repositorio y los 64 commits de historia.

Qué hacer: transferir el proyecto a un grupo del área, no a una persona. Y hacer un clon de respaldo *antes* de cualquier movimiento:

```
git clone --mirror https://172.16.196.12/mvazquez/tih.git tih-respaldo.git
```

2.4 Llave SSH al servidor

Hay una llave `ed25519` autorizada en `~/.ssh/authorized_keys` de `mvazquez@172.16.83.21`, con comentario `tih-deploy-mariana`. Quítenla cuando ella salga y pongan la de quien reciba.

3. Accesos y credenciales

No hay secretos en el repositorio y no deben agregarse. Todo se configura por variables de entorno en archivos `.env` que están en `.gitignore`.

Pidan que los valores se entreguen por un gestor de contraseñas o canal seguro, **nunca por correo ni dentro del repo**.

3.1 Inventario de accesos

#	Acceso	Para qué	Dónde se configura
1	**SQL Server** `172.16.200.99\UMBRACODB`, base `cmde_public`	Fuente de todos los datos	`backend/.env`
2	**Servidor de aplicación** `mvazquez@172.16.83.21` (SSH + `sudo`)	Desplegar	Llave SSH + contraseña de `sudo`
3	**GitLab interno** `172.16.196.12`	Código	Usuario GitLab
4	**PostgreSQL** `localhost:5433`, base `tih_pg`	Solo migración (no producción)	`postgres_migration/.env`
5	**Panel** `#admin` del sitio	Estadísticas de uso	`ADMIN_KEY` en `backend/.env`

3.2 Claves que espera `backend/.env`

Copien `backend/.env.example` a `backend/.env` y llenen. La plantilla está en el repo; **los valores no**.

Variable	Notas
`DB_SERVER`, `DB_INSTANCE`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SCHEMA`	SQL Server. Lo esencial.
`DB_ENGINE`	`mssql` (actual) o `postgres`. **No cambiar** sin leer §7.1.
`PG_HOST`, `PG_PORT`, `PG_DATABASE`, `PG_USER`, `PG_PASSWORD`	Solo si `DB_ENGINE=postgres`.
`CORS_ORIGINS`	En producción: `http://172.16.83.21:8091`
`SECRET_KEY`	Cambiar en producción.
`ADMIN_KEY`	Clave del panel `#admin`. **El `.env` del servidor y el local son distintos.**
`DOCS_ENABLED`	`false` en producción (oculta `/docs`).

> Todas las variables tienen valor por defecto en `backend/app/config.py`, así que un `.env` incompleto **no impide arrancar** — usa los defaults. Es cómodo y peligroso a la vez: verifiquen que `ADMIN_KEY` no se haya quedado en el default.

3.3 El `.env` de producción

Vive en `/opt/tih/backend/.env`, **fuera** de `app/`. Los despliegues copian solo `app/`, así que nunca se pisa. No lo borren ni lo sobrescriban.

4. Arquitectura

4.1 Lo que corre de verdad

> El `README.md` dice **Nginx** y describe carpetas que ya no existen. Está desactualizado. Lo de abajo está verificado contra el servidor el 30/sep/2026.



`nginx/nginx.conf` sigue en el repo pero **no se usa**: en el servidor Nginx está inactivo y quien sirve es Apache (`deploy/apache_tih.conf`).

4.2 Stack

Capa	Tecnología
Backend	Python 3.13, FastAPI, SQLAlchemy (solo motor de conexión, **sin ORM**), `pyodbc`, Pydantic
Frontend	React 18, Vite, Tailwind CSS, Apache ECharts
Estado (front)	Zustand — `frontend/src/store/useExplorer.js`
Producción	Apache + gunicorn/uvicorn + systemd. Sin Docker.
Chat (en pausa)	Ollama local, modelo `qwen2.5:7b` — ver §7.2

5. Arrancar en local

Requisitos: Python 3.11+, Node 18+, y el **driver ODBC de SQL Server** (`ODBC Driver 17/18 for SQL Server`) — sin eso el backend no conecta.

```
cd backend && pip install -r requirements.txt && uvicorn app.main:app --reload
```

```
cd frontend && npm install && npm run dev
```

- Backend: http://localhost:8000 · Frontend: http://localhost:5173
- Antes de arrancar: copiar `backend/.env.example` a `backend/.env` y llenarlo.

La primera petición al Panorama tarda 1–2 minutos reconstruyendo caché. No está colgado. Después va rápido.

> En la raíz hay un `package.json`, `package-lock.json` y `node_modules/` que son basura de un `npm install` mal ubicado. El frontend real está en `frontend/`. Se pueden borrar.

6. Base de datos

El esquema completo está en `DATABASE_SCHEMA.md`, y qué endpoint usa qué tabla en `DATABASE_DEPENDENCIES.md`. Consúltenlos antes de tocar cualquier consulta.

Lo que hay que saber

- **TIH es de solo lectura**, salvo tres excepciones donde sí escribe:
  - `dbo.cache_meta` y `dbo.produccion_pozos` — caché materializado por `services/pozos_cache.py`.
  - `PRECIOS_AUTOM` — la crea el backend si no existe; guarda el precio del día.
- **No hay ORM.** Todo es SQL escrito a mano dentro de `backend/app/services/*.py`. Si el equipo de datos renombra una tabla o columna, **hay que corregir el SQL a mano**. Nada avisa.
- `WHERE ID <> 1` aparece en varias consultas sobre `PRODUCCION_CAMPOS`: excluye una fila de totales/placeholder. No lo quiten sin confirmar.
- Las columnas de negocio vienen en MAYÚSCULAS y mezclan español e inglés (`PETROLEO_MBD`, `ASIG_CONT`, `FECHA`, `CUENCA`). Herencia del origen.

7. Pendientes abiertos

7.1 Migración a PostgreSQL

Estado: avanzada, validada en desarrollo, sin cortar a producción.

> Esto es un **proyecto aparte**, no una dependencia del sitio. Producción sigue y seguirá leyendo de SQL Server hasta que alguien decida lo contrario. Se puede dejar parado indefinidamente sin consecuencias.

Los 8 dominios están extraídos, transformados y validados contra una instancia de prueba (`localhost:5433`). Se decidió **no** reescribir los cargadores del equipo de datos, sino construir un puente que sincroniza SQL Server -> Postgres (ver §2.2). El backend ya soporta ambos motores mediante `DB_ENGINE`.

- Plan y arquitectura: `postgres_migration/00_PLAN.md`
- Bitácora y validaciones: `postgres_migration/MIGRATION_STATUS.md`
- Estado operativo: `BD MIGRACIÓN PENDIENTE DE HACER SERVER.md`

Falta: repetir el proceso contra la instancia Postgres definitiva (la actual es de prueba) y mover el puente a un servidor.

7.2 Chatbot

Estado: funciona en local, pausado antes de producción.

Implementado con Ollama (`qwen2.5:7b`). No se desplegó porque falta decidir dónde correr el modelo. Detalle en `CHATBOT_PENDIENTE.md`.

7.3 Menores

- El **reporte PDF del Panorama** conserva un diseño tipo infografía mientras el resto de descargables se aplanó. Quedó sin decidir si se unifica.
- `README.md` desactualizado (ver §4.1).
- `nginx/nginx.conf` sin uso.
- Respaldos borrables en el servidor cuando haya confianza: `/opt/tih/backend/app.bak.despliegue` y `/opt/tih/frontend/dist.bak`.

8. Despliegue

Procedimiento **verificado el 30/sep/2026**. Las advertencias vienen de errores que ya ocurrieron.

8.1 Compilar

```
cd frontend && npm run build
```

Anoten el nombre del bundle:

```
grep -o 'assets/index-[A-Za-z0-9_-]*\.js' frontend/dist/index.html
```

8.2 Subir

No hagan `scp` directo a `/opt` — es de root y falla. Suban a la carpeta personal y muevan después.

```
scp -r frontend/dist mvazquez@172.16.83.21:~/tih_dist_nuevo
```

```
cd backend && tar czf /tmp/app.tgz --exclude='__pycache__' app && scp /tmp/app.tgz mvazquez@172.16.83.21:~/
```

8.3 Verificar ANTES de mover

Este paso no es opcional. Una vez se movió el `dist` bueno a respaldo sin comprobar que el nuevo había llegado, y **el sitio se cayó**.

```
ssh mvazquez@172.16.83.21 'grep -o "assets/index-[A-Za-z0-9_-]*\.js" ~/tih_dist_nuevo/index.html'
```

Debe coincidir con el bundle que compilaron.

8.4 Instalar

Frontend:

```
ssh -t mvazquez@172.16.83.21 'cd /opt/tih/frontend && rm -rf dist.bak && mv dist dist.bak && mv ~/tih_dist_
nuevo dist && chmod -R a+rX dist'
```

Backend:

```
ssh -t mvazquez@172.16.83.21 'cd /opt/tih/backend && rm -rf ~/app_nuevo && mkdir ~/app_nuevo && tar xzf ~/a
pp.tgz -C ~/app_nuevo && cd ~/app_nuevo/app && python3 -m compileall -q . && cd /opt/tih/backend && rm -rf
app.bak.despliegue && cp -a app app.bak.despliegue && rm -rf app && mv ~/app_nuevo/app ./app && sudo system
ctl restart tih_backend && sleep 8 && systemctl is-active tih_backend'
```

`chmod -R a+rX` **no es opcional**: el `scp` deja la carpeta solo para el usuario y Apache (`www-data`) responde **Forbidden**.

El frontend **no requiere reiniciar nada**; el backend **sí**.

8.5 Comprobar

```
ssh mvazquez@172.16.83.21 'curl -s http://127.0.0.1:8091/ | grep -o "assets/index-[A-Za-z0-9_-]*\.js"'
```

No basta con que responda 200: **verifiquen que el bundle servido sea el nuevo**. El primer `/resumen/` tras reiniciar tarda 1–2 min.

8.6 Revertir

```
ssh -t mvazquez@172.16.83.21 'cd /opt/tih && rm -rf frontend/dist && mv frontend/dist.bak frontend/dist &&
cd backend && rm -rf app && mv app.bak.despliegue app && sudo systemctl restart tih_backend'
```

8.7 Trampas conocidas

- **Comparen por checksum, no por fechas.** Una vez el servidor llevaba **17 archivos de backend atrasados** sin que se notara; los `mtime` engañan porque `tar` los preserva. Para saber qué difiere de verdad:

```
cd backend && find app -name "*.py" -exec md5sum {} \; | sed 's/ \*/  /' | awk '{print $2" "$1}' | sort > /
tmp/l.txt && ssh mvazquez@172.16.83.21 'cd /opt/tih/backend && find app -name "*.py" -exec md5sum {} \;' |
awk '{print $2" "$1}' | sort > /tmp/s.txt && join /tmp/l.txt /tmp/s.txt | awk '$2!=$3 {print $1}'
```

- **No ejecuten el comando de reversión "por si acaso".** Deshace el despliegue que acaba de funcionar. Pasó dos veces seguidas y costó un buen rato entender por qué el código nuevo "no se aplicaba".
- **PowerShell vs bash:** `&&` no es separador válido en PowerShell y `ssh-copy-id` no existe ahí. Lo más seguro es `ssh -t usuario@host 'comando'` con comillas **simples**, para que PowerShell no expanda `$(...)` ni `*`.

9. Mapa del código

Backend — `backend/app/`

```
main.py            FastAPI, CORS, cabeceras de seguridad, registro de routers
config.py          Settings (pydantic-settings), lee backend/.env
database.py        Conexión a SQL Server / Postgres según DB_ENGINE
models/schemas.py  Esquemas Pydantic
routers/           Un router por dominio — solo validan y delegan
services/          Toda la lógica y el SQL. Aquí vive el trabajo real.
  cache_utils.py     Decorador @cached() — caché EN MEMORIA, se pierde al reiniciar
  pozos_cache.py     Excepción: materializa una tabla real en la BD
  rate_limit.py      Límite de peticiones del panel de uso
```

Patrón: el router valida con Pydantic y delega; el service construye el SQL y lo ejecuta. Si buscan dónde se calcula algo, es en `services/`.

Frontend — `frontend/src/`

```
api/          Clientes HTTP (client.js)
store/        useExplorer.js — estado global (Zustand). El corazón de la app.
components/
  Charts/       ChartArea.jsx     (decide qué vista mostrar)
                DynamicTable.jsx  (la tabla jerárquica)
  Fichas/       Tarjetas de detalle de campo/pozo/operador/asignación
  Header/       TopBar.jsx (filtros, periodo), BuscadorGlobal.jsx
  UI/           ErrorBoundary y utilidades
utils/        formato.js (reglas numéricas), nombreCampo.js, export*.js
```

Los dos archivos difíciles son `ChartArea.jsx` y `DynamicTable.jsx`. Concentran la lógica de jerarquías, plegado y exportación. **Están muy comentados y los comentarios explican el *porqué*, no el *qué*** — léanlos antes de cambiar algo ahí; muchos documentan errores ya cometidos.

Particularidades que conviene conocer

- **Formato numérico**: `utils/formato.js` es el único lugar. Regla: entero -> sin decimales; con parte decimal -> exactamente 2. Nunca un `.00` colgado.
- **Nombres de entidad**: `utils/nombreCampo.js` normaliza MAYÚSCULAS a título, pero **los códigos de contrato y asignación se conservan en mayúsculas** (`CNH-A4.OGARRIO/2018`, `A-0203-4M`).
- **Fichas**: clic izquierdo abre la ficha si está libre; si ya hace otra cosa (plegar, seleccionar), la ficha va en **clic derecho**.
- **Caché en memoria**: reiniciar el backend lo vacía y la primera consulta vuelve a tardar. Es normal, no es una falla.

10. Guía de la documentación existente

Hay ~30 archivos `.md` en la raíz. **Casi todos son bitácoras de sesión**, útiles como arqueología pero no como manual. Orden de lectura sugerido:

Prioridad	Archivo	Qué es
1	`TRASPASO.md`	Este documento
2	`CLAUDE.md`	Convenciones del proyecto. Conciso y vigente.
3	`DATABASE_SCHEMA.md`	Esquema de tablas y columnas
4	`DATABASE_DEPENDENCIES.md`	Qué endpoint usa qué tabla
5	`deploy/DESPLIEGUE.md`	Instalación desde cero en un servidor nuevo
6	`postgres_migration/MIGRATION_STATUS.md`	Estado de la migración
7	`CHATBOT_PENDIENTE.md`	El chat en pausa
—	`CONSULTAS_SQL_SIH.md`	Consultas de referencia del sistema anterior
—	`CONTEXTO_SESION_*.md`, `SESION_*.md`	**Bitácoras.** No son documentación formal. Sirven para responder "¿por qué está hecho así?". El más reciente es `CONTEXTO_SESION_TABLAS_EXPORTS_DESPLIEGUE.md`.
	`README.md`	**Desactualizado** (ver §4.1)

11. Primeros pasos para quien recibe

1. Pedir los accesos de §3.1 y clonar el repositorio.
2. Instalar el driver ODBC y levantar en local (§5).
3. Leer `CLAUDE.md` y `DATABASE_SCHEMA.md`.
4. **Reubicar lo que estaba local (§2)**, empezando por 2.3 (transferir el repositorio), que es lo único con urgencia real. Lo demás puede ordenarse con calma.
5. Hacer un cambio mínimo de frontend y desplegarlo siguiendo §8, para probar el circuito completo con red de seguridad.